🐍 🧬 💖

Git & GitHub para Bioinformática

Do zero ao primeiro Pull Request: teoria e prática, no terminal e no VS Code

Dra. Natalia Coutouné
Outubro, 2026

CC BY-NC-SA 4.0

Roteiro

1

Por que controlar versões?

Teoria

2

Preparando o ambiente

Teoria + prática

3

Fluxo local: add e commit

Prática



Intervalo

4

Trabalhando com o GitHub

Prática

5

Branches e colaboração

Prática

6

Fechamento e exercício final

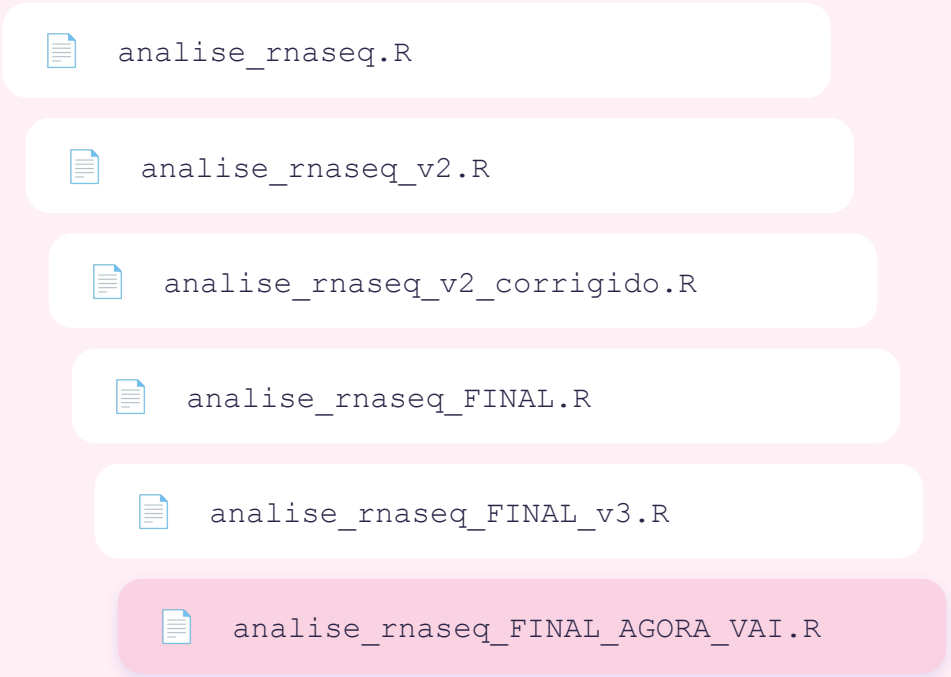
Prática

PARTE 1

Por que controlar versões?

O problema, os conceitos e o vocabulário que vamos usar o dia todo

Sem controle de versão, o histórico vira uma pasta cheia de "final_v3"



Perguntas sem resposta

- Qual versão gerou a Figura 2 do artigo?
- O que mudou entre a v2 e a FINAL?
- Quem alterou o filtro de qualidade?
- Como juntar as mudanças de duas pessoas sem perder nada?

O controle de versão guarda cada estado do projeto e deixa você voltar no tempo



Histórico

Cada mudança fica registrada com autor, data e motivo.



Voltar no tempo

Recupere qualquer versão anterior de um arquivo ou do projeto inteiro.



Trabalho em paralelo

Várias pessoas (ou ideias) avançam ao mesmo tempo sem se atropelar.

🧬 Em bioinformática isso é reprodutibilidade: saber exatamente qual código gerou cada resultado publicado.

Git é a ferramenta no seu computador; GitHub é o site que hospeda os repositórios



Git

programa · roda localmente

- Funciona offline, no seu computador
- Guarda o histórico na pasta oculta .git/
- Comandos: init, add, commit, branch…
- Criado em 2005 por Linus Torvalds



GitHub

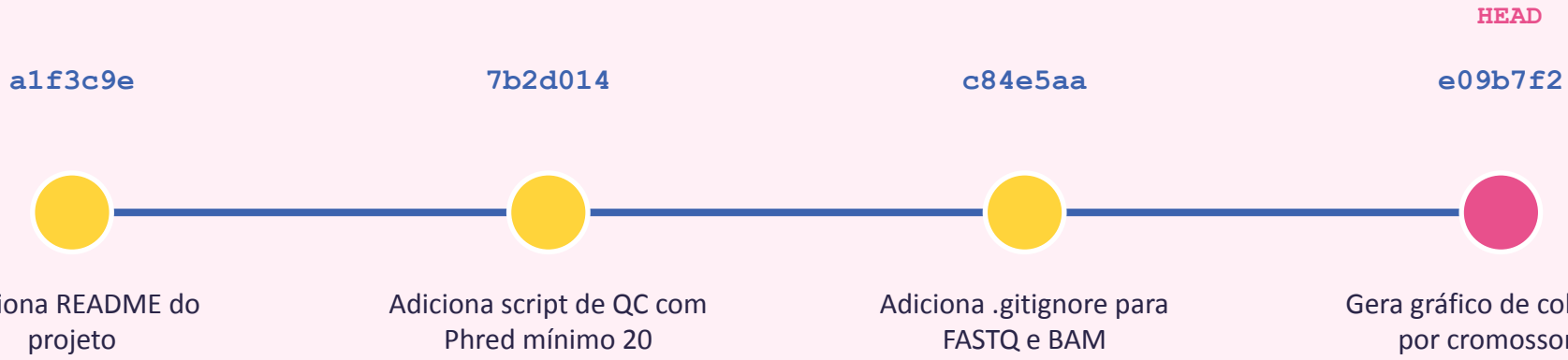
plataforma · na internet

- Guarda uma cópia do repositório na nuvem
- Colaboração: Pull Requests, revisão, Issues
- Portfólio público do seu código
- Alternativas: GitLab, Bitbucket, Codeberg

Seu código passa por quatro lugares, e cada comando move mudanças entre eles

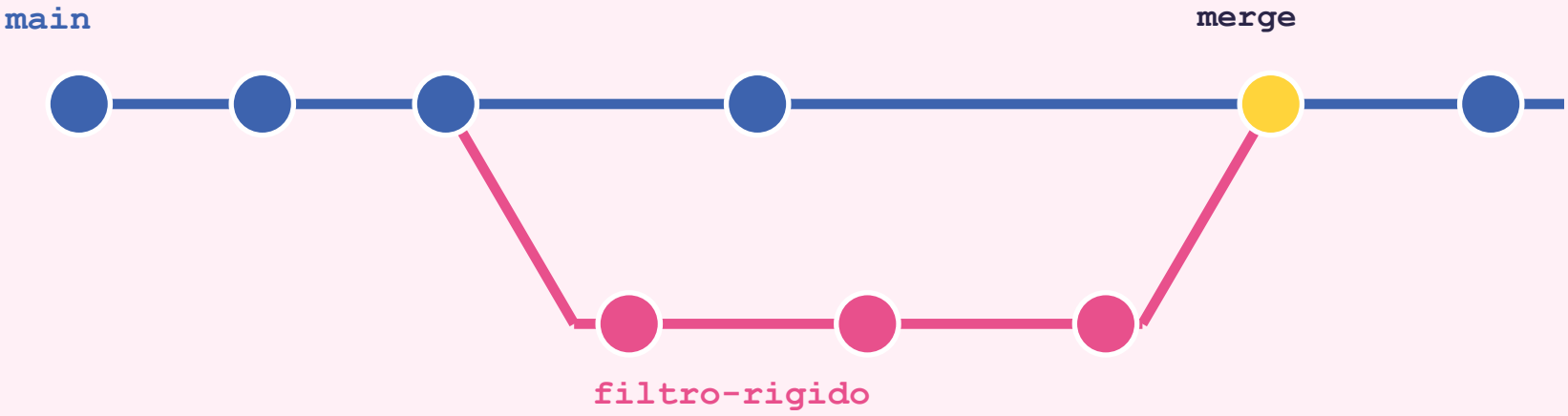


Um commit é uma foto do projeto, com autor, data e uma mensagem explicando o porquê



Cada commit guarda: hash (identificador único) · autor e e-mail · data · mensagem · ponteiro para o commit anterior

Branches deixam você testar uma ideia sem mexer na versão que funciona



`main` é a versão estável. Cada experimento ganha sua branch e volta pela `merge` quando estiver pronto.

PARTE 2

Preparando o ambiente

Git, VS Code, conta no GitHub e a autenticação

Antes de tudo: confira a instalação e apresente-se ao Git

Checklist

- ✓ Git instalado
- ✓ VS Code instalado
- ✓ Conta no GitHub criada
- ✓ VS Code conectado ao GitHub (Sign in)
- ✓ Nome e e-mail configurados

```
# confere a instalação
$ git --version

# uma vez por computador
$ git config --global user.name "Seu Nome"
$ git config --global user.email "voce@email.com"
$ git config --global init.defaultBranch main
$ git config --global core.editor "code --wait"

# conferir
$ git config --list
```

O GitHub não aceita a senha da conta no terminal: use o VS Code, um token ou SSH



1 · Pelo VS Code

Recomendado para a aula

Ícone de conta (canto inferior esquerdo) ⟶ Sign in with GitHub. O VS Code guarda a credencial e o push funciona.



2 · Token (PAT)

Necessário no Colab

GitHub ⟶ Settings ⟶ Developer settings ⟶ Personal access tokens. O token substitui a senha.



3 · Chave SSH

Ótimo para servidores e clusters

ssh-keygen -t ed25519, depois cole a chave pública em Settings ⟶ SSH and GPG keys.

PARTE 3

Fluxo local: editar, add, commit

Cada passo primeiro no terminal, depois no VS Code

Nosso projeto de exemplo: uma análise de RNA-seq de levedura

```
rnaseq-levedura/
├── README.md
├── .gitignore
├── data/
│   └── amostra_teste.fastq    # ignorado
├── scripts/
│   └── qc_reads.py
└── results/                   # ignorado
```

```
$ mkdir rnaseq-levedura && cd rnaseq-levedura
$ mkdir data scripts results
$ git init
```

O que o qc_reads.py faz

- Lê um arquivo FASTQ
- Calcula a qualidade média (Phred) de cada read
- Marca como DESCARTAR as reads abaixo de QUALIDADE_MINIMA = 20
- No VS Code: File → Open Folder → aba Source Control → Initialize Repository

O ciclo do dia a dia tem três passos: editar, escolher (add) e salvar (commit)

Ação	Terminal	VS Code (Source Control, Ctrl+Shift+G)
Ver o que mudou	`git status`	Letras ao lado dos arquivos: U, M, D
Ver a diferença	`git diff`	Clique no arquivo: diff lado a lado
Preparar (staging)	`git add arquivo`	Botão + (Stage Changes)
Tirar do staging	`git restore --staged arq`	Botão − (Unstage Changes)
Salvar uma versão	`git commit -m "msg"`	Escreva a mensagem → ✓ Commit
Ver o histórico	`git log --oneline --graph`	Seção Graph ou Timeline no Explorer

git status e o VS Code mostram a mesma informação, cada um do seu jeito

```
$ git status
On branch main
Changes to be committed:
        new file:   scripts/qc_reads.py
Changes not staged for commit:
        modified:   README.md
Untracked files:
        notas.txt
```

- **U** — Untracked: arquivo novo, o Git ainda não acompanha
- **M** — Modified: arquivo alterado desde o último commit
- **A** — Added: novo arquivo já no staging
- **D** — Deleted: arquivo apagado

O .gitignore é uma lista de padrões: o Git finge que esses arquivos não existem

O que é

- Um arquivo de texto chamado .gitignore, na raiz do projeto
- Cada linha é um padrão de nome de arquivo ou pasta
- O que combina com um padrão some do git status e não entra no git add
- Ele também vai no commit, e assim vale para toda a equipe

Padrão	O que o Git ignora
`# texto`	Nada: é um comentário
`*.bam`	Todo arquivo terminado em .bam, em qualquer pasta
`results/`	A pasta results inteira
`data/raw/*.fastq`	Só os .fastq dentro de data/raw
`!pipeline.log`	Exceção: mesmo com *.log na lista, este fica

Já estava no Git? O .gitignore não vale para arquivos já commitados. Use **`git rm --cached arquivo`**

O .gitignore mantém dados brutos, resultados e senhas fora do repositório

```
# Dados brutos e grandes
*.fastq
*.fastq.gz
*.bam
data/raw/
# Resultados regeneráveis
results/
# Segredos
.env
# Lixo de sistema
__pycache__/
.DS_Store
```

Dados brutos

FASTQ e BAM têm gigabytes. Guarde no SRA/ENA, Zenodo ou servidor e versione o script que os baixa.

Resultados

Se o script gera, não precisa versionar a saída: versione o script.

Segredos

Tokens, senhas e chaves nunca entram. Uma vez no GitHub, considere vazado.

Boas mensagens de commit explicam o que mudou para o seu eu do futuro

✅ Assim

```
Filtra reads com Phred < 20 no QC

Corrige caminho do genoma de referência

Adiciona gráfico de cobertura

Remove amostra contaminada S07
```

❌ Assim não

```
update

mudanças

agora vai

asdfgh
```

Dica: complete a frase "Este commit…" com um verbo no presente. Uma ideia por commit.

Mão na massa 1: crie o projeto e faça três commits

1. Crie a pasta rnaseq-levedura e rode git init
2. Crie o README.md → add → commit
3. Crie scripts/qc_reads.py e data/amostra_teste.fastq
4. Crie o .gitignore e confira com git status que o FASTQ sumiu
5. Commit do script + .gitignore
6. Mude uma linha, veja o git diff e faça o terceiro commit pelo VS Code

⏱️ **20 minutos · Material: caderno Colab (Parte A) e cola de comandos**

PARTE 4

Trabalhando com o GitHub

remote, push, pull, clone e editores no navegador

Para publicar no GitHub, ligue o repositório local ao remoto e faça push

```
# 1. crie o repositório vazio no GitHub
# 2. ligue o local ao remoto
$ git remote add origin \
  https://github.com/USUARIO/rnaseq-levedura.git
$ git remote -v

# 3. primeiro envio
$ git push -u origin main

# depois, no dia a dia
$ git push        # envia
$ git pull        # recebe
```

No VS Code

- Publish Branch: cria o repositório no GitHub e faz o primeiro push
- Sync Changes: faz pull e push de uma vez
- Barra inferior: setas mostram commits a enviar ↑ e a receber ↓
- Regra de ouro: pull antes de começar, push ao terminar

git clone copia um repositório inteiro, com todo o histórico, para o seu computador

```
$ git clone https://github.com/USUARIO/REPOSITORIO.git
$ cd REPOSITORIO
$ git log --oneline
```

No VS Code

- Ctrl+Shift+P ⟶ Git: Clone
- Cole a URL e escolha a pasta
- Clique em Open para abrir o projeto

Mão na massa 2 · 15 min

- Publique o rnaseq-levedura no GitHub
- Edite o README pelo site e faça pull
- Clone o repositório de uma colega

Sem instalar nada: github.dev para editar, Codespaces para o fluxo completo

	github.dev	GitHub Codespaces
Como abrir	Tecla . (ponto) no repositório	Code → Codespaces → Create
O que é	VS Code no navegador	Máquina Linux na nuvem + VS Code
Terminal	❌ Não tem	✅ git, python, R, conda
Custo	Grátis	Cota grátis mensal, depois pago
Ideal para	Editar um README ou script	Rodar e testar o fluxo inteiro

⚠️ **Ao terminar, pare ou apague o codespace em github.com/codespaces para não gastar a cota.**

PARTE 5

Branches e colaboração

branch, merge, conflitos e Pull Requests

Criar, trocar e juntar branches leva quatro comandos ou alguns cliques

Ação	Terminal	VS Code
Listar branches	`git branch`	Clique no nome da branch (barra inferior)
Criar e entrar	`git switch -c filtro-rigido`	Barra inferior → Create new branch…
Trocar	`git switch main`	Barra inferior → escolha a branch
Juntar na atual	`git merge filtro-rigido`	Ctrl+Shift+P → Git: Merge Branch…
Apagar	`git branch -d filtro-rigido`	Ctrl+Shift+P → Git: Delete Branch…
Enviar ao GitHub	`git push -u origin filtro-rigido`	Publish Branch

Um conflito aparece quando duas branches mudam a mesma linha, e você decide o que fica

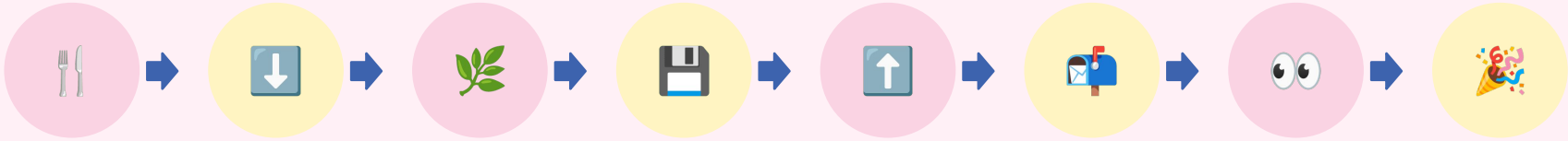
```
$ git merge filtro-rigido
CONFLICT (content): Merge conflict
  in scripts/qc_reads.py

<<<<<<< HEAD
QUALIDADE_MINIMA = 25
=======
QUALIDADE_MINIMA = 30
>>>>>>> filtro-rigido
```

Como resolver

1. Abra o arquivo em conflito (marcado com ! na aba Source Control)
2. Escolha: Accept Current, Accept Incoming ou Accept Both (ou edite à mão)
3. Apague as marcas <<<<<<< ======= >>>>>>>
4. git add arquivo → git commit

O Pull Request é como o GitHub organiza a colaboração: propor, revisar, juntar



Fork × Clone

Fork é uma cópia do repositório na sua conta do GitHub. Clone é uma cópia no seu computador.

Por que PR?

Alguém revisa o código antes de ele entrar na main. Erros aparecem cedo e a equipe aprende junto.

Mão na massa 3: em dupla, provoquem um conflito e resolvam juntas

A	Adiciona B como colaboradora (Settings → Collaborators)
B	Clona o repositório de A
A e B	Mudam QUALIDADE_MINIMA para valores diferentes e fazem commit
A	Faz push primeiro ✅
B	Tenta push ❌ → git pull → resolve o conflito no VS Code → push ✅

⏱️ **20 minutos**

PARTE 6

Fechamento

Como desfazer erros, boas práticas e o exercício final

Socorro, errei! Quase tudo no Git tem volta, se você usar o comando certo

Situação	Comando
Descartar mudanças não commitadas num arquivo	git restore arquivo
Tirar um arquivo do staging	git restore --staged arquivo
Corrigir a mensagem do último commit (antes do push)	git commit --amend -m "nova msg"
Desfazer um commit que já foi enviado	git revert HASH
Ver um arquivo como era no passado	git show HASH:arquivo
Parar de rastrear um arquivo que devia ser ignorado	git rm --cached arquivo

⚠️ **Evite git reset --hard e git push --force até ganhar prática: eles apagam histórico.**

Seis hábitos que deixam um projeto de bioinformática reprodutível

Commits pequenos

Uma ideia por commit, com mensagem clara

README

Diga o que o projeto faz e como rodar

Ambiente versionado

environment.yml ou requirements.txt no repositório

Dados fora do Git

SRA/ENA, Zenodo ou servidor, mais o script que baixa

Nada de segredos

Senhas e tokens em .env, listado no .gitignore

Licença

MIT ou GPL para o código poder ser reutilizado

Exercício final: cada pessoa abre um Pull Request no repositório da turma

```
# 1. Fork no GitHub, depois:
$ git clone https://github.com/SEU-USUARIO/repo-da-turma.git
$ cd repo-da-turma
$ git switch -c adiciona-SEU-NOME
# 2. crie alunos/SEU-NOME.md
$ git add alunos/SEU-NOME.md
$ git commit -m "Adiciona apresentação de SEU NOME"
$ git push -u origin adiciona-SEU-NOME
# 3. No GitHub: Compare & pull request
```

No arquivo

- Seu nome e área de pesquisa
- O organismo ou dado com que você trabalha 🧬
- Um comando de Git que aprendeu hoje
- Depois: comente o PR de uma colega 💬

Para continuar estudando depois da aula



Pro Git (em português)

`git-scm.com/book/pt-br/v2`



Learn Git Branching

`learngitbranching.js.org`



GitHub Skills

`skills.github.com`



Software Carpentry: Git

`swcarpentry.github.io/git-novice`



Oh Shit, Git!?

`ohshitgit.com/pt_BR`



Cola de comandos da aula

`arquivo cola-git-github.md`

Obrigada! 💖

Dúvidas?

`git add .` · `git commit -m "aprendi git hoje"` · `git push`

nat.coutoune@gmail.com
@cest__nat